Санкт-Петербургский государственный университет

Горошко Андрей Витальевич

Выпускная квалификационная работа

Исследование влияния дообучения большой языковой модели на качество классификации инструментов в AI-агенте

Уровень образования: бакалавриат

Направление

Основная образовательная программа *СВ.5001.2022 «Математическое обеспечение и администрирование информационных систем»*

Научный руководитель:
Доцент кафедры математики и компьютерных наук Якубович Ю. В.

Рецензент:
должность, ООО «Место работы», степень (если есть), Рецензент Р. Р.

Санкт-Петербург
2026

Оглавление

1. Введение **4**

2. Постановка задачи **7**

2.1. Формальная постановка 7

2.2. Гипотезы исследования 8

2.3. Исследовательские вопросы 8

2.4. Критерии успеха 9

2.5. Ограничения исследования 10

3. Обзор литературы **11**

3.1. Большие языковые модели и архитектура Transformer . . 11

3.2. Методы дообучения больших языковых моделей 12

3.3. Гипотеза поверхностного выравнивания 14

3.4. AI-агенты и выбор инструментов 15

3.5. Синтетическая генерация обучающих данных 16

3.6. Выводы обзора 17

4. Математические основы **19**

4.1. Формализация задачи бинарной классификации 19

4.2. Метрики качества классификации 20

4.3. Низкоранговая адаптация (LoRA) 21

4.4. Статистические критерии сравнения классификаторов . 23

4.5. Бутстрап доверительные интервалы 26

5. Методология экспериментов **27**

5.1. Пайплайн дообучения и оценки 27

5.2. Описание бенчмарков 28

5.3. Обучающие датасеты 29

5.4. Протокол дообучения 31

5.5. Дизайн эксперимента 32

6. Результаты экспериментов и их анализ **35**

6.1. Оценка базовой модели 35

6.2. Основные результаты дообучения 36
6.3. Валидация на реальных данных 37
6.4. Анализ кривой обучения 38
6.5. Анализ типа обучающих данных 39
6.6. Стабильность дообучения 40
6.7. Анализ ошибок 41
6.8. Статистическая значимость 42
6.9. Ответы на исследовательские вопросы 44

7. Программная реализация 46
7.1. Архитектура системы 46
7.2. Подсистема статистического анализа 47
7.3. Подсистема инференса 48
7.4. Подсистема генерации данных 49
7.5. Обеспечение качества кода 49
7.6. Технологический стек 49

8. Заключение 51
8.1. Направления дальнейших исследований 53

Список литературы 55

1. Введение

Одним из центральных направлений развития систем на основе больших языковых моделей (LLM) является создание автономных агентов, способных взаимодействовать с внешними инструментами — API, базами данных, поисковыми системами — для решения задач, выходящих за пределы генерации текста. Подход, при котором модель самостоятельно определяет, какой инструмент вызвать и с какими параметрами, получил название *tool use* (или *function calling*) и активно исследуется в работах [21, 7, 20]. Качество выбора инструмента непосредственно определяет применимость агента в промышленных системах: ошибка классификации приводит либо к некорректному ответу, либо к полному отсутствию результата.

В настоящей работе рассматривается AI-агент в сфере недвижимости, обрабатывающий пользовательские запросы на естественном языке. Агент должен классифицировать каждый запрос в один из двух классов и вызвать соответствующую функцию: `ResolveEntities` для географических объектов (класс GEO) и `SearchTags` для характеристик объектов недвижимости (класс TAG). Базовая модель YandexGPT 5.1 Pro, используемая в качестве ядра агента, демонстрирует на объединённом бенчмарке из 500 примеров точность (accuracy) лишь 81,8% при значении Macro $F_1 = 0{,}526$. Анализ ошибок выявляет систематический сдвиг (bias): из 174 запросов класса TAG верно классифицированы лишь 49,4%, а для 34 запросов модель вообще не генерирует вызов инструмента. Таким образом, базовая модель обладает выраженным предпочтением класса GEO, что делает её непригодной для промышленного применения без дополнительной адаптации.

Центральная гипотеза данного исследования состоит в том, что выбор инструмента по своей природе является элементом *стиля ответа* модели, а не следствием недостатка знаний о предметной области. Эта позиция опирается на гипотезу поверхностного выравнивания (Superficial Alignment Hypothesis), сформулированную в работе [8]: знания и способности LLM формируются на этапе предобучения, тогда как дообу-

чение (fine-tuning) корректирует лишь формат и стиль ответов. Аналогичные наблюдения о разграничении знаний и стиля приведены в [6]. Если данная гипотеза верна применительно к задаче tool selection, то даже небольшой объём обучающих примеров, дообученных методом LoRA [11], должен радикально улучшить качество классификации, не требуя масштабного расширения обучающей выборки.

Целью работы является экспериментальная проверка указанной гипотезы. Для этого решаются следующие задачи:

1. формализация задачи классификации инструментов как задачи бинарной классификации с двумя бенчмарками (GEO и TAG);

2. разработка инфраструктуры для дообучения и систематической оценки моделей;

3. проведение серии из 12 экспериментов с вариацией типа обучающих данных (GEO, TAG, смешанные) и объёма выборки;

4. статистический анализ результатов с применением критериев Кохрена Q и Макнемара для попарных сравнений моделей.

Научная новизна работы заключается в систематическом экспериментальном исследовании влияния параметров дообучения на качество tool selection в контексте конкретной промышленной модели (YandexGPT). В отличие от существующих работ, сосредоточенных преимущественно на англоязычных моделях открытого доступа, данное исследование проводится на коммерческой модели с русскоязычным доменом и включает строгое статистическое сравнение 12 моделей на двух независимых бенчмарках. Основной результат — достижение точности 99,4% лучшей дообученной моделью при всего одной ошибке на 500 примерах — подтверждает гипотезу о стилистической природе tool selection и демонстрирует практическую эффективность подхода.

Работа организована следующим образом. В разделе 2 дана формальная постановка задачи. Раздел 3 содержит обзор литературы по tool use, методам дообучения и гипотезе поверхностного выравнивания.

В разделе 4 описаны теоретические основы используемых методов. Раздел 5 посвящён методологии экспериментов, раздел 6 — анализу результатов. В разделе 7 описана программная реализация. Раздел 8 содержит заключение и направления дальнейших исследований.

2. Постановка задачи

2.1. Формальная постановка

Пусть $\mathcal{X}$ — пространство пользовательских запросов на естественном языке, поступающих AI-агенту в предметной области поиска объектов недвижимости. Определим конечное множество инструментов (tools):

$$\mathcal{T} = \{t_{\text{GEO}},\ t_{\text{TAG}}\},$$

где $t_{\text{GEO}} \equiv$ `ResolveEntities` — инструмент разрешения географических сущностей (город, район, улица, адрес), а $t_{\text{TAG}} \equiv$ `SearchTags` — инструмент поиска по характеристикам объекта (площадь, тип ремонта, этажность, количество комнат).

Определение 1. *Классификатором первого вызова инструмента называется отображение*

$$f_\theta \colon \mathcal{X} \to \mathcal{T},$$

реализуемое большой языковой моделью (LLM) с параметрами θ, которое каждому входному запросу $x \in \mathcal{X}$ ставит в соответствие инструмент $t \in \mathcal{T}$, выбираемый моделью в качестве первого `tool_call` *в генерируемом ответе.*

В качестве базовой линии (baseline) для сравнения используется модель YandexGPT 5.1 Pro (f_{θ_0}), не подвергавшаяся дообучению. Дообученные модели f_{θ^*} получаются из YandexGPT Lite — компактной версии семейства YandexGPT — методом LoRA (Low-Rank Adaptation) через Yandex Cloud API.

Введём ошибки классификации в терминах проверки статистических гипотез. Примем за нулевую гипотезу H_0: «запрос x относится к классу GEO».

Определение 2. Ошибка I рода *(ложноположительное срабатывание t_{GEO}): запрос класса TAG классифицирован как GEO, то есть $f_\theta(x) = t_{\text{GEO}}$ при истинной метке $y(x) = t_{\text{TAG}}$. Такая ошибка приводит к тому,*

что агент пытается найти несуществующий географический объект вместо применения фильтра по характеристикам.

Определение 3. Ошибка II рода *(ложноположительное срабатывание t_{TAG}): запрос класса GEO классифицирован как TAG, то есть $f_\theta(x) = t_{\text{TAG}}$ при истинной метке $y(x) = t_{\text{GEO}}$. Такая ошибка приводит к потере географической привязки в результатах поиска.*

Дополнительно выделяется третий исход: модель не генерирует `tool_call` вовсе, что является критическим сбоем в контексте AI-агента. Обозначим долю таких ответов как $\rho_\varnothing$.

2.2. Гипотезы исследования

Гипотеза 1. *Дообучение модели YandexGPT Lite методом LoRA статистически значимо улучшает accuracy классификации первого `tool_call` на бенчмарке `benchmark-combined` по сравнению с базовой моделью YandexGPT 5.1 Pro (accuracy = 81,8%) до уровня, превышающего 90% (при уровне значимости $\alpha = 0{,}05$, критерий Макнемара).*

Гипотеза 2. *Качество обучающих данных (сбалансированный микс реальных пользовательских запросов и синтетических примеров) оказывает большее влияние на итоговое качество классификации, чем объём обучающей выборки. В частности, модель, обученная на меньшем, но качественно подготовленном датасете, превосходит модель, обученную на большем, но несбалансированном датасете.*

Гипотеза 3. *Процесс дообучения через Yandex Cloud API является нестабильным: идентичные запуски обучения с одними и теми же гиперпараметрами и данными приводят к статистически значимому разбросу итоговых метрик классификации.*

2.3. Исследовательские вопросы

Для систематической проверки сформулированных гипотез определим следующие исследовательские вопросы:

RQ1. *Каково влияние дообучения на метрики классификации?* — Количественная оценка изменений accuracy, macro F1, precision и recall по классам t_{GEO} и t_{TAG} между базовой моделью f_{θ_0} и дообученными моделями f_{θ^*} на бенчмарках `benchmark-combined` (500 примеров: 326 GEO / 174 TAG) и `benchmark-real` (250 примеров: 203 GEO / 47 TAG).

RQ2. *Как зависит качество классификации от объёма обучающих данных?* — Анализ зависимости метрик от размера обучающей выборки при фиксированном составе и качестве данных.

RQ3. *Как тип обучающих данных влияет на робастность модели?* — Сравнение моделей, обученных на синтетических данных, реальных пользовательских запросах и их комбинациях, с точки зрения обобщающей способности и устойчивости к изменению распределения входных данных.

RQ4. *Статистически значимы ли различия между моделями?* — Применение критериев Макнемара и bootstrap-доверительных интервалов для попарного сравнения моделей.

2.4. Критерии успеха

Критерии успеха определяются на основании baseline-метрик базовой модели YandexGPT 5.1 Pro и целевых значений, обоснованных требованиями прикладной системы. Результаты сведены в Таблице 1.

Таблица 1: Критерии успеха: baseline-метрики базовой модели и целевые значения

Метрика	**Baseline**	**Целевое значение**	**Бенчмарк**
Accuracy	81,8%	$> 90\%$	combined
Macro F1	0,526	$> 0{,}85$	combined
Recall (t_{TAG})	49,4%	$> 75\%$	combined
Доля без `tool_call` ($\rho_{\varnothing}$)	6,8%	$< 3\%$	combined

Низкий baseline-показатель Recall для класса t_{TAG} (49,4%) означает, что базовая модель классифицирует почти каждый второй TAG-запрос как GEO, что является основной проблемой и мотивацией для дообучения. Значение Macro F1 = 0,526 отражает существенный дисбаланс качества между классами: высокий Recall для класса GEO при критически низком Recall для TAG.

2.5. Ограничения исследования

Сформулируем ключевые ограничения, в рамках которых проводится данное исследование:

1. **Фиксированная архитектура модели.** Исследование ограничено семейством моделей YandexGPT 5.1 (Lite и Pro). Результаты не обязаны переноситься на модели других архитектур и провайдеров.

2. **Ограниченный контроль над процессом обучения.** Дообучение производится через Yandex Cloud API, что ограничивает доступ к гиперпараметрам обучения (learning rate, rank LoRA-адаптеров, число эпох регулируется косвенно). Внутренние механизмы оптимизации являются непрозрачными.

3. **Бинарная постановка.** Рассматривается только два инструмента из множества инструментов реальной системы. Вопрос масштабирования на $|\mathcal{T}| > 2$ выходит за рамки работы.

4. **Дисбаланс классов.** Оба бенчмарка содержат существенное преобладание класса GEO (65% в `combined`, 81% в `real`), что отражает реальное распределение пользовательских запросов, но ограничивает статистическую мощность выводов по классу TAG.

5. **Невоспроизводимость на уровне API.** Yandex Cloud API не гарантирует детерминированность генерации при идентичных параметрах, что вносит дополнительный источник дисперсии в результаты.

3. Обзор литературы

В настоящем разделе систематизируются основные результаты, относящиеся к предметной области исследования: архитектура больших языковых моделей, методы их параметрически эффективного дообучения, гипотеза поверхностного выравнивания, а также подходы к построению AI-агентов, способных использовать внешние инструменты. Обзор формирует теоретическую базу для последующей постановки эксперимента и интерпретации результатов.

3.1. Большие языковые модели и архитектура Transforme

Архитектура Transformer, предложенная в работе Vaswani и др. [1], стала фундаментом современных больших языковых моделей (LLM). Ключевым нововведением этой архитектуры является механизм самовнимания (self-attention), позволяющий модели при обработке каждого токена учитывать контекст всех остальных токенов входной последовательности. В отличие от рекуррентных нейронных сетей, обрабатывающих последовательность последовательно, механизм самовнимания допускает полную параллелизацию вычислений, что существенно ускоряет обучение на больших корпусах текстов. Оригинальная архитектура Transformer включает кодировщик (encoder) и декодировщик (decoder), однако в дальнейшем получили широкое распространение модели, использующие только одну из этих компонент.

Семейство моделей GPT (Generative Pre-trained Transformer), описанное в работе Radford и др. [9], использует архитектуру на основе исключительно декодировщика (decoder-only). Модель обучается авторегрессионно: на каждом шаге она предсказывает следующий токен последовательности, обусловливая предсказание на всех предшествующих токенах. Работа GPT-2 [9] продемонстрировала, что языковые модели, обученные на больших неразмеченных корпусах, способны решать широкий спектр задач обработки естественного языка без явного обучения с учителем (zero-shot), что обосновало концепцию языкового моделирования как универсального подхода к многозадачному обуче-

нию.

Семейство моделей YandexGPT [22] разработано компанией Яндекс и представляет собой ряд генеративных языковых моделей различного масштаба (YandexGPT Lite, YandexGPT Pro). Эти модели доступны через API Yandex Cloud и поддерживают дообучение (fine-tuning) на пользовательских данных, что делает их пригодной платформой для экспериментального исследования влияния дообучения на специализированные задачи, включая классификацию инструментов в контексте AI-агентов.

3.2. Методы дообучения больших языковых моделей

Полное дообучение (full fine-tuning) предобученной языковой модели предполагает обновление всех её параметров на целевом наборе данных. Для моделей, содержащих миллиарды параметров, такой подход сопряжён с рядом практических трудностей: высокие вычислительные затраты, необходимость хранения отдельной копии всех параметров для каждой решаемой задачи, а также риск катастрофического забывания (catastrophic forgetting) — явления, при котором модель утрачивает ранее приобретённые знания в ходе обучения на новых данных [13].

Проблема катастрофического забывания была систематически исследована в работе Kirkpatrick и др. [13], где было показано, что нейронные сети при последовательном обучении на различных задачах склонны к значительной деградации качества на ранее освоенных задачах. Авторы предложили метод Elastic Weight Consolidation (EWC), основанный на регуляризации, которая ограничивает изменение параметров, наиболее важных для ранее решённых задач.

В ответ на указанные ограничения полного дообучения был разработан класс методов, объединённых под общим названием **Parameter-Efficient Fine-Tuning (PEFT)**. Обзор данного направления представлен в работе Ding и др. [14], опубликованной в Nature Machine Intelligence. Авторы систематизируют существующие подходы к параметрически

эффективному дообучению и выделяют несколько основных стратегий: добавление небольших обучаемых модулей к фиксированной модели, модификация лишь малого числа существующих параметров, а также обучение дополнительных представлений на входе модели.

Метод **LoRA** (Low-Rank Adaptation), предложенный в работе Hu и др. [11], основан на гипотезе о низкоранговой природе обновлений весов при дообучении. Вместо обновления полной матрицы весов $W \in \mathbb{R}^{d\times k}$ метод параметризует приращение через произведение двух матриц низкого ранга:

$$W' = W + \Delta W = W + BA, \quad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d,k). \quad (1)$$

При этом исходная матрица W остаётся замороженной, а обучаются только матрицы A и B, содержащие в совокупности $r(d+k)$ параметров — существенно меньше, чем dk параметров полной матрицы. На практике выбор r в диапазоне от 4 до 64 позволяет обучать менее 1% от общего числа параметров модели, сохраняя при этом качество, сопоставимое с полным дообучением [11].

Метод **Prompt Tuning**, предложенный Lester и др. [10], реализует альтернативную стратегию: вместо модификации параметров модели к входной последовательности токенов добавляются обучаемые непрерывные векторы (soft prompts), которые оптимизируются под конкретную задачу. При этом все параметры самой модели остаются замороженными. Авторы показали, что при увеличении масштаба модели эффективность Prompt Tuning приближается к полному дообучению, при значительно меньшем числе обучаемых параметров.

Подход **Adapter Layers**, предложенный Houlsby и др. [15], предполагает встраивание небольших дополнительных нейросетевых модулей (адаптеров) между слоями предобученной модели. Адаптеры обычно имеют архитектуру «бутылочного горлышка» (bottleneck): понижение размерности, нелинейная активация, повышение размерности. Обучаются только параметры адаптеров, что позволяет использовать одну замороженную базовую модель для множества задач, переключаясь между ними путём замены адаптеров.

3.3. Гипотеза поверхностного выравнивания

Важным вопросом при дообучении языковых моделей является характер изменений, которые оно вносит: приобретает ли модель принципиально новые знания или лишь корректирует форму выдачи уже усвоенной при предобучении информации?

В работе Zhou и др. [8] была сформулирована **гипотеза поверхностного выравнивания** (Superficial Alignment Hypothesis). Авторы продемонстрировали, что модель LIMA, дообученная всего на 1000 тщательно отобранных примерах, способна генерировать ответы высокого качества, сопоставимые с результатами моделей, обученных на значительно больших наборах данных с использованием обучения с подкреплением на основе обратной связи от человека (RLHF). Согласно данной гипотезе, практически все фактические знания модель приобретает на этапе предобучения, тогда как дообучение с инструкциями (instruction fine-tuning) преимущественно обучает модель *формату и стилю* взаимодействия с пользователем. Иными словами, выравнивание является «поверхностным»: оно затрагивает не глубинные знания модели, а лишь способ их представления.

Эти выводы согласуются с результатами работы Gudibande и др. [6], в которой было показано, что модели, дообученные на выходах более мощных проприетарных моделей (так называемая имитация), улучшают стиль генерации, но не приобретают фактических знаний, присущих моделям-учителям. Модели-имитаторы демонстрируют улучшение качества ответов по субъективным оценкам (стиль, связность), однако на объективных бенчмарках, измеряющих фактическую корректность, прирост отсутствует [6].

Для настоящего исследования гипотеза поверхностного выравнивания имеет ключевое значение. Если дообучение преимущественно корректирует стиль, а не знания, то выбор инструмента в контексте AI-агента может быть интерпретирован как элемент «стиля» генерации — устойчивый паттерн оформления ответа, который модель способна усвоить из относительно небольшого числа качественных примеров. Данная

интерпретация обосновывает ожидание того, что целенаправленное дообучение на компактном, но качественном наборе данных способно значимо улучшить точность классификации инструментов.

3.4. AI-агенты и выбор инструментов

Понятие AI-агента на основе большой языковой модели подразумевает систему, в которой LLM выступает в роли центрального «мыслящего» компонента, способного планировать действия, обращаться к внешним инструментам и интерпретировать результаты их выполнения. Обзорная работа Xi и др. [17] систематизирует архитектуры таких агентов и выделяет три ключевых компонента: модуль восприятия (perception), модуль планирования и рассуждений (brain), а также модуль действия (action), включающий взаимодействие с внешними инструментами.

Одним из наиболее влиятельных подходов к организации рассуждений агента является фреймворк **ReAct**, предложенный Yao и др. [16]. ReAct объединяет два процесса: генерацию цепочки рассуждений (reasoning) и выполнение действий (acting) в чередующемся цикле «мысль — действие — наблюдение» (thought–action–observation). Модель последовательно формулирует рассуждение о текущей ситуации, выполняет действие (например, вызов API или поисковый запрос), получает наблюдение (результат действия) и использует его для следующего шага рассуждений. Такая архитектура позволяет агенту адаптивно выбирать инструменты на каждом шаге, исходя из контекста задачи.

В работе Schick и др. [21] был предложен подход **Toolformer**, в котором языковая модель самостоятельно обучается вставлять вызовы внешних инструментов (калькулятор, поисковая система, переводчик и др.) в генерируемый текст. Обучение осуществляется в формате self-supervised: модель генерирует кандидатные вызовы инструментов, фильтрует их по критерию снижения перплексии и дообучается на отфильтрованных данных. Toolformer продемонстрировал, что модели способны научиться определять, *когда* и *какой* инструмент следует использовать, без явной разметки.

Работа Patil и др. [7] непосредственно связана с задачей настоящего исследования. Авторы предложили модель **Gorilla**, дообученную для корректного вызова API из обширного каталога. Gorilla обучалась на наборе данных, сгенерированном с помощью Self-Instruct, и продемонстрировала способность точно генерировать вызовы API, превосходя по этому показателю модели GPT-4 без специализированного дообучения. Данный результат подтверждает гипотезу о том, что целенаправленное дообучение на задачу выбора и вызова инструментов способно существенно повысить качество классификации.

Обзорная работа Qin и др. [20] систематизирует направление обучения моделей использованию инструментов (tool learning). Авторы выделяют четыре ключевых этапа взаимодействия модели с инструментами: понимание задачи, планирование использования инструментов, непосредственный вызов инструмента и обработка его результата. Особый интерес представляет выделенный авторами этап *выбора инструмента* (tool selection) — задача классификации, в которой модель должна из доступного набора инструментов выбрать наиболее подходящий для решения текущей подзадачи. Именно эта задача является предметом настоящего исследования.

3.5. Синтетическая генерация обучающих данных

Подготовка качественных обучающих данных для дообучения языковых моделей является ресурсоёмкой задачей. Метод **Self-Instruct**, предложенный Wang и др. [18], предлагает подход к автоматической генерации обучающих примеров с использованием самой языковой модели. Процедура включает итеративную генерацию инструкций, входных данных и эталонных ответов с последующей фильтрацией некачественных и дублирующихся примеров. Авторы продемонстрировали, что дообучение модели на синтетически сгенерированных данных приводит к значимому улучшению качества следования инструкциям, приближаясь к результатам моделей, обученных на данных, размеченных вручную.

Результаты Self-Instruct [18] согласуются с выводами LIMA [8]: если выравнивание является поверхностным и затрагивает преимущественно формат генерации, то для его достижения достаточно относительно небольшого числа качественных примеров, даже если эти примеры получены синтетическим путём. Для задачи классификации инструментов это означает, что эффективный обучающий набор может быть сформирован с помощью LLM-генерации при условии обеспечения разнообразия сценариев и корректности разметки. Модель Gorilla [7] является практическим подтверждением данного подхода: её обучающий набор был получен методом Self-Instruct.

3.6. Выводы обзора

Проведённый обзор литературы позволяет сформулировать следующие ключевые наблюдения.

Во-первых, архитектура Transformer [1] и авторегрессионные decoder-only модели [9] являются технологическим фундаментом современных LLM, включая семейство YandexGPT [22], используемое в настоящей работе.

Во-вторых, методы параметрически эффективного дообучения, в особенности LoRA [11], позволяют адаптировать модели к специализированным задачам при минимальных вычислительных затратах и сниженном риске катастрофического забывания [13, 14].

В-третьих, гипотеза поверхностного выравнивания [8, 6] обосновывает ожидание того, что дообучение способно значимо улучшить паттерны генерации (включая выбор инструмента), не требуя при этом масштабных обучающих наборов.

В-четвёртых, задача выбора инструмента в контексте AI-агентов [17, 16, 20] является активной областью исследований, а работы Toolformer [21] и Gorilla [7] демонстрируют эффективность дообучения для её решения.

Вместе с тем, **открытым остаётся вопрос** о том, каким образом дообучение модели масштаба YandexGPT Lite влияет на качество классификации инструментов в реальной агентной системе, каков мини-

мальный объём обучающих данных, достаточный для значимого улучшения, и в какой мере наблюдаемый эффект согласуется с гипотезой поверхностного выравнивания. Настоящая работа направлена на экспериментальное исследование именно этих вопросов.

4. Математические основы

В настоящем разделе вводятся формальные определения и математический аппарат, необходимые для строгой постановки задачи классификации инструментов, оценки качества моделей и статистического сравнения результатов.

4.1. Формализация задачи бинарной классификации

Пусть $\mathcal{X}$ — пространство признаков, элементами которого являются текстовые запросы пользователей к AI-агенту. Определим множество меток $\mathcal{Y} = \{0, 1\}$, где метка 0 соответствует классу `TAG` (тегирование), а метка 1 — классу `GEO` (геокодирование).

Определение 4 (Классификатор)**.** *Классификатором называется измеримое отображение $f_\theta \colon \mathcal{X} \to \mathcal{Y}$, параметризованное вектором параметров $\theta \in \Theta$, где Θ — пространство параметров модели.*

Пусть дана выборка $\{(x_i, y_i)\}_{i=1}^n$, где $x_i \in \mathcal{X}$, $y_i \in \mathcal{Y}$. Обозначим предсказания классификатора $\hat{y}_i = f_\theta(x_i)$.

Определение 5 (Матрица ошибок)**.** *Матрицей ошибок (confusion matrix) классификатора f_θ на выборке $\{(x_i, y_i)\}_{i=1}^n$ называется набор из четырёх величин:*

$$\mathrm{TP} = \big|\{i : \hat{y}_i = 1 \ \wedge \ y_i = 1\}\big|, \tag{2}$$

$$\mathrm{FP} = \big|\{i : \hat{y}_i = 1 \ \wedge \ y_i = 0\}\big|, \tag{3}$$

$$\mathrm{TN} = \big|\{i : \hat{y}_i = 0 \ \wedge \ y_i = 0\}\big|, \tag{4}$$

$$\mathrm{FN} = \big|\{i : \hat{y}_i = 0 \ \wedge \ y_i = 1\}\big|, \tag{5}$$

где $|\cdot|$ обозначает мощность множества. Очевидно, $\mathrm{TP} + \mathrm{FP} + \mathrm{TN} + \mathrm{FN} = n$.

На основании матрицы ошибок введём понятия ошибок первого и второго рода, интерпретация которых определяется выбором «положительного» класса. В нашей задаче положительным классом является `GEO` ($y = 1$).

Определение 6 (Ошибка I рода). *Ошибкой первого рода («ложная тревога») называется условная вероятность ошибочного отнесения объекта класса* `TAG` *к классу* `GEO`*:*

$$\alpha = P\big(f_\theta(x) = 1 \mid y = 0\big) = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}}. \tag{6}$$

Определение 7 (Ошибка II рода). *Ошибкой второго рода («пропуск цели») называется условная вероятность ошибочного отнесения объекта класса* `GEO` *к классу* `TAG`*:*

$$\beta = P\big(f_\theta(x) = 0 \mid y = 1\big) = \frac{\mathrm{FN}}{\mathrm{FN} + \mathrm{TP}}. \tag{7}$$

Заметим, что $1 - \beta$ есть мощность (power) критерия, совпадающая в терминологии классификации с полнотой (recall) для положительного класса.

4.2. Метрики качества классификации

Пусть $\mathcal{C} = \{0, 1\}$ — множество классов, $|\mathcal{C}| = 2$. Для каждого класса $c \in \mathcal{C}$ обозначим через TP_c, FP_c, TN_c, FN_c элементы матрицы ошибок, построенной в стратегии «один против всех» (one-vs-rest) для класса c.

Определение 8 (Accuracy). *Доля правильных ответов (accuracy) классификатора определяется как*

$$\mathrm{Acc} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}}. \tag{8}$$

Определение 9 (Precision). *Точность (precision) для класса $c \in \mathcal{C}$ определяется как*

$$P_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c}. \tag{9}$$

Величина P_c характеризует долю истинных представителей класса c среди всех объектов, отнесённых классификатором к классу c.

Определение 10 (Recall). *Полнота (recall) для класса $c \in \mathcal{C}$ определяется как*

$$R_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c}. \tag{10}$$

Величина R_c характеризует долю верно распознанных представителей класса c среди всех объектов, истинно принадлежащих классу c.

Определение 11 (F_1-мера)**.** *F_1-мера для класса c определяется как среднее гармоническое точности и полноты:*

$$F_{1,c} = \frac{2\,P_c\,R_c}{P_c + R_c} = \frac{2\,\mathrm{TP}_c}{2\,\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}. \tag{11}$$

Для обобщения метрик на многоклассовый случай (а также для единообразного представления результатов бинарной классификации) используются агрегированные варианты F_1-меры.

Определение 12 (Macro F_1)**.** *Макро-усреднённая F_1-мера определяется как среднее арифметическое F_1-мер по всем классам:*

$$F_1^{\mathrm{macro}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} F_{1,c}. \tag{12}$$

Данная метрика придаёт равный вес каждому классу вне зависимости от его представленности в выборке.

Определение 13 (Weighted F_1)**.** *Взвешенная F_1-мера определяется как*

$$F_1^{\mathrm{weighted}} = \sum_{c \in \mathcal{C}} \frac{n_c}{N}\, F_{1,c}, \tag{13}$$

где $n_c = |\{i : y_i = c\}|$ — число объектов класса c в выборке, $N = \sum_{c \in \mathcal{C}} n_c$ — общий объём выборки. Эта метрика учитывает дисбаланс классов.

Систематический анализ метрик классификации и их свойств приведён в [19].

4.3. Низкоранговая адаптация (LoRA)

Метод низкоранговой адаптации (Low-Rank Adaptation, LoRA) [11] позволяет эффективно дообучать предобученные языковые модели, модифицируя лишь малую часть параметров.

Пусть $W_0 \in \mathbb{R}^{d \times k}$ — матрица весов некоторого линейного слоя предобученной модели. В процессе полного дообучения (full fine-tuning)

вычисляется обновление $\Delta W \in \mathbb{R}^{d\times k}$, после чего используется матрица $W = W_0 + \Delta W$. Количество обучаемых параметров при этом составляет $d \cdot k$.

Ключевая идея LoRA состоит в параметризации обновления через произведение двух матриц малого ранга:

$$W = W_0 + \Delta W = W_0 + B\,A, \tag{14}$$

где $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, а $r \ll \min(d,k)$ — ранг адаптации.

Определение 14 (Низкоранговая адаптация)**.** *Низкоранговой адаптацией с рангом r называется замена обновления $\Delta W \in \mathbb{R}^{d\times k}$ на произведение $\Delta W = B\,A$, где $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, при фиксированной исходной матрице W_0.*

Количество обучаемых параметров при использовании LoRA для одного слоя составляет

$$r\,(d+k) \tag{15}$$

вместо $d \cdot k$. При $r \ll \min(d,k)$ достигается значительное сокращение числа параметров:

$$\frac{r\,(d+k)}{d\cdot k} = r\left(\frac{1}{k} + \frac{1}{d}\right) \ll 1. \tag{16}$$

Инициализация матриц выполняется следующим образом:

$$A_{ij} \sim \mathcal{N}(0,\sigma^2), \qquad B = \mathbf{0}_{d\times r}, \tag{17}$$

что гарантирует $\Delta W = BA = \mathbf{0}$ на момент начала обучения. Таким образом, дообучение начинается с исходной предобученной модели без внесения начальных возмущений.

Прямой проход через адаптированный слой для входного вектора $h \in \mathbb{R}^k$ вычисляется как

$$h' = W_0\,h + \frac{\alpha}{r}\,B\,A\,h, \tag{18}$$

где $\alpha > 0$ — масштабирующий гиперпараметр, контролирующий вклад

адаптации. Множитель α/r обеспечивает стабильность обучения при изменении ранга r.

4.4. Статистические критерии сравнения классификаторов

Для статистически обоснованного сравнения нескольких классификаторов, обученных на одних и тех же данных и оценённых на общей тестовой выборке, необходимо применять специализированные критерии, учитывающие зависимость предсказаний.

4.4.1. Критерий Q Кохрена

Критерий Q Кохрена [3] применяется для проверки гипотезы об однородности $k \geqslant 2$ бинарных классификаторов, оценённых на одной и той же выборке из n объектов.

Пусть $X_{ij} \in \{0, 1\}$ — индикатор правильного ответа j-го классификатора ($j = 1, \ldots, k$) на i-м объекте ($i = 1, \ldots, n$). Введём обозначения:

$$T_j = \sum_{i=1}^{n} X_{ij}, \qquad R_i = \sum_{j=1}^{k} X_{ij}, \qquad \bar{T} = \frac{1}{k} \sum_{j=1}^{k} T_j. \tag{19}$$

Здесь T_j — количество правильных ответов j-го классификатора, R_i — количество классификаторов, давших правильный ответ на i-й объект.

Определение 15 (Нулевая гипотеза критерия Кохрена)**.** *H_0: все k классификаторов имеют одинаковую вероятность правильного ответа, т. е. $P(X_{i1} = 1) = P(X_{i2} = 1) = \ldots = P(X_{ik} = 1)$ для всех i.*

Статистика критерия Кохрена определяется формулой

$$Q = \frac{k(k-1) \sum\limits_{j=1}^{k} (T_j - \bar{T})^2}{k \sum\limits_{i=1}^{n} R_i - \sum\limits_{i=1}^{n} R_i^2}. \tag{20}$$

При справедливости H_0 и достаточно большом n статистика Q имеет асимптотическое распределение хи-квадрат:

$$Q \xrightarrow{d} \chi^2(k-1). \tag{21}$$

Гипотеза H_0 отвергается при $Q > \chi^2_{1-\alpha}(k-1)$, где $\chi^2_{1-\alpha}(k-1)$ — $(1-\alpha)$-квантиль распределения χ^2 с $k-1$ степенями свободы. Отвержение H_0 означает, что хотя бы один классификатор статистически значимо отличается от остальных по доле правильных ответов.

4.4.2. Критерий Макнемара

Для попарного сравнения двух классификаторов A и B, оценённых на одной и той же выборке из n объектов, применяется критерий Макнемара [12, 4].

Построим таблицу сопряжённости 2×2 по результатам работы двух классификаторов:

	B верно	B неверно
A верно	n_{11}	n_{10}
A неверно	n_{01}	n_{00}

(22)

где n_{10} — количество объектов, на которых A ответил верно, а B — неверно; n_{01} — количество объектов, на которых B ответил верно, а A — неверно. Величины n_{11} и n_{00} (согласованные ответы) не несут информации о различии классификаторов.

Определение 16 (Нулевая гипотеза критерия Макнемара)**.** *H_0: вероятности дискордантных исходов равны, т. е.*

$$P(A \text{ верно, } B \text{ неверно}) = P(B \text{ верно, } A \text{ неверно}). \tag{23}$$

При малых значениях $n_{10} + n_{01}$ (что типично для задач с ограниченными тестовыми выборками) используется точный биномиальный тест. При справедливости H_0 случайная величина n_{10} имеет биномиальное распределение:

$$n_{10} \sim \mathrm{Bin}\left(n_{10} + n_{01},\ \tfrac{1}{2}\right). \tag{24}$$

p-значение вычисляется как вероятность наблюдения столь же или более экстремального результата:

$$p = 2\,\min\Big(P\big(X \leqslant n_{10}\big),\; P\big(X \geqslant n_{10}\big)\Big), \quad X \sim \mathrm{Bin}\big(n_{10} + n_{01},\; \tfrac{1}{2}\big)\,. \tag{25}$$

При больших $n_{10} + n_{01}$ может быть использовано асимптотическое приближение со статистикой

$$\chi^2_{\mathrm{McN}} = \frac{(n_{10} - n_{01})^2}{n_{10} + n_{01}}, \tag{26}$$

имеющей при H_0 распределение $\chi^2(1)$.

4.4.3. Поправка Бонферрони

При проведении m попарных сравнений возникает проблема множественных проверок: вероятность хотя бы одного ложноположительного результата (family-wise error rate, FWER) возрастает с ростом m.

Определение 17 (FWER)**.** *Групповая вероятность ошибки первого рода определяется как*

$$\mathrm{FWER} = P\big(\textit{хотя бы одно ложное отвержение } H_0\big). \tag{27}$$

Поправка Бонферрони [2] является простейшим способом контроля FWER. При проведении m одновременных проверок скорректированный уровень значимости каждого отдельного теста полагается равным

$$\alpha_{\mathrm{adj}} = \frac{\alpha}{m}, \tag{28}$$

где α — желаемый уровень FWER. Неравенство Бонферрони гарантирует, что

$$\mathrm{FWER} = P\left(\bigcup_{i=1}^{m} A_i\right) \leqslant \sum_{i=1}^{m} P(A_i) = m \cdot \alpha_{\mathrm{adj}} = \alpha, \tag{29}$$

где A_i — событие ложного отвержения H_0 в i-м тесте.

В нашем случае при сравнении k классификаторов попарно число сравнений составляет $m = \binom{k}{2} = \frac{k(k-1)}{2}$.

4.5. Бутстрап доверительные интервалы

Для оценки неопределённости метрик качества классификации применяется метод непараметрического бутстрапа [5], не требующий предположений о параметрическом виде распределения метрики.

Пусть $\mathcal{D} = \{(x_i, y_i, \hat{y}_i)\}_{i=1}^n$ — тестовая выборка с истинными и предсказанными метками. Пусть $\theta = T(\mathcal{D})$ — значение интересующей метрики (например, F_1, accuracy), вычисленное на выборке $\mathcal{D}$.

Определение 18 (Бутстрап-выборка)**.** *Бутстрап-выборкой $\mathcal{D}_b^*$ ($b = 1, \ldots, B$) называется выборка объёма n, полученная из $\mathcal{D}$ случайным выбором с возвращением:*

$$\mathcal{D}_b^* = \big\{(x_{i_1}, y_{i_1}, \hat{y}_{i_1}),\ \ldots,\ (x_{i_n}, y_{i_n}, \hat{y}_{i_n})\big\}, \quad i_1, \ldots, i_n \overset{\text{iid}}{\sim} \text{Uniform}\{1, \ldots, n\}. \tag{30}$$

Алгоритм построения бутстрап доверительного интервала перцентильным методом состоит из следующих шагов.

1. Для каждого $b = 1, \ldots, B$ сформировать бутстрап-выборку $\mathcal{D}_b^*$ и вычислить $\theta_b^* = T(\mathcal{D}_b^*)$.
2. Упорядочить бутстрап-оценки: $\theta_{(1)}^* \leqslant \theta_{(2)}^* \leqslant \ldots \leqslant \theta_{(B)}^*$.
3. Построить доверительный интервал уровня $(1 - \alpha)$ как

$$\text{CI}_{1-\alpha} = \Big[\theta_{(\lfloor B\alpha/2 \rfloor)}^*,\ \theta_{(\lceil B(1-\alpha/2) \rceil)}^*\Big]. \tag{31}$$

В настоящей работе используется $B = 1000$ бутстрап-выборок и уровень доверия $1 - \alpha = 0{,}95$, что обеспечивает построение 95% доверительных интервалов для всех исследуемых метрик.

Обоснование состоятельности перцентильного метода основано на принципе подстановки (plug-in principle): эмпирическое распределение $\hat{F}_n$ аппроксимирует истинное распределение F, а распределение $T(\mathcal{D}^*) - T(\mathcal{D})$ аппроксимирует распределение $T(\mathcal{D}) - \theta$, где θ — истинное значение метрики. При $n \to \infty$ эта аппроксимация является состоятельной в силу теоремы Гливенко–Кантелли.

5. Методология экспериментов

В данном разделе описывается методология экспериментальной части исследования: пайплайн дообучения и оценки моделей, состав бенчмарков и обучающих датасетов, протокол дообучения, а также статистический дизайн эксперимента.

5.1. Пайплайн дообучения и оценки

Экспериментальный процесс организован в виде шестиэтапного пайплайна, обеспечивающего воспроизводимость и систематичность проведения экспериментов. Общая схема пайплайна представлена на Рис. 1.

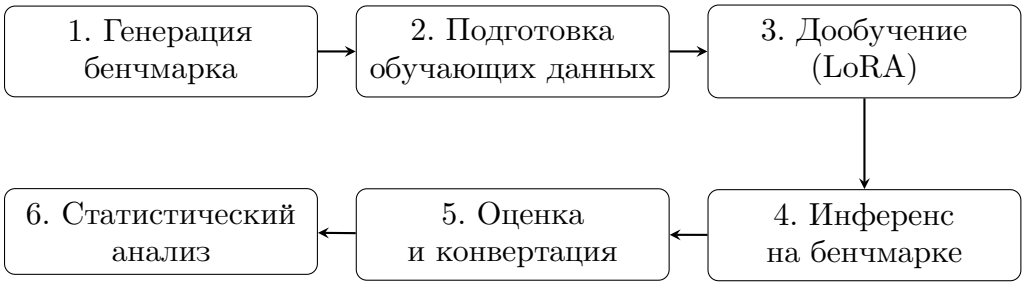

Рис. 1: Пайплайн дообучения и оценки моделей.

Каждый этап пайплайна выполняет следующие функции:

1. **Генерация бенчмарка.** Формирование оценочных наборов данных двух типов: смешанного (реальные и синтетические запросы) и реального (только пользовательские запросы). Каждый пример содержит пользовательский запрос и эталонную метку инструмента (GEO или TAG).

2. **Подготовка обучающих данных.** Создание обучающих датасетов различного объёма, типа и классового баланса. Синтетические данные генерируются по методологии Self-Instruct [18] с последующей гуманизацией для приближения к стилистике реальных запросов.

3. **Дообучение (LoRA).** Адаптация базовой модели YandexGPT

Lite методом LoRA [11] через API Yandex Cloud [22]. На выходе — дообученная модель с уникальным идентификатором чекпоинта.

4. **Инференс на бенчмарке.** Последовательный прогон каждого примера из бенчмарка через модель с фиксированными параметрами генерации. Результат — XML-разметка с вызовом инструмента.

5. **Оценка и конвертация.** Парсинг ответов модели, извлечение имени вызванного инструмента из тегов `<tool_call>`, сопоставление с эталонной меткой. Вычисление метрик accuracy, precision, recall, F1 для каждого класса.

6. **Статистический анализ.** Применение непараметрических тестов (Cochran Q, McNemar) и бутстрап-оценок доверительных интервалов для проверки статистической значимости различий между моделями.

5.2. Описание бенчмарков

Для оценки качества классификации инструментов используются два бенчмарка, различающихся по составу и назначению (Табл. 2).

Таблица 2: Характеристики оценочных бенчмарков.

Бенчмарк	Объём	GEO	TAG	Тип данных
`benchmark-combined`	500	326 (65,2%)	174 (34,8%)	Реальные + синтет
`benchmark-real`	250	203 (81,2%)	47 (18,8%)	Только реальн

benchmark-combined (500 примеров) содержит смесь реальных пользовательских запросов и синтетических примеров. Соотношение классов GEO/TAG составляет 326/174, что отражает естественное преобладание географических запросов в предметной области. Данный бенчмарк предназначен для оценки общей способности модели к классификации инструментов.

benchmark-real (250 примеров) состоит исключительно из реальных пользовательских запросов, собранных из логов AI-агента. Распределение классов существенно смещено: 203 запроса GEO (81,2%) против 47 запросов TAG (18,8%). Данный бенчмарк предназначен для оценки качества модели в условиях, максимально приближённых к промышленной эксплуатации.

5.2.1. Политика разметки пограничных случаев

При создании бенчмарков применялась единая политика разметки, определяющая отнесение запроса к классу GEO или TAG в неоднозначных ситуациях:

- **GEO**: запрос содержит конкретные имена собственные — названия станций метро («ст. м. Сокол»), улиц («ул. Тверская»), районов, городов, конкретных адресов. Критерий — наличие именованной географической сущности, требующей разрешения через геокодер.

- **TAG**: запрос описывает класс инфраструктуры или характеристику объекта без привязки к конкретному имени собственному. Примеры: «рядом с парком», «с камином», «у метро» (как категория транспортной доступности, без указания конкретной станции).

Ключевое разграничение: выражение «у метро Сокол» классифицируется как GEO (содержит имя собственное), тогда как «у метро» — как TAG (описывает характеристику без конкретной привязки).

5.3. Обучающие датасеты

Для исследования влияния объёма и типа обучающих данных на качество классификации были подготовлены шесть основных обучающих датасетов (Табл. 3).

Таблица 3: Характеристики обучающих датасетов.

Датасет	Объём	Тип данных	GEO	TAG	Гумани
`train-synthetic-50`	50	Синтетические	54,0%	46,0%	Д
`train-synthetic-250`	250	Синтетические	53,6%	46,4%	Д
`train-synthetic-500`	500	Синтетические	55,8%	44,2%	Д
`train-synthetic-1000`	908	Синтетические	52,0%	48,0%	Д
`train-real-250`	250	Реальные	82,4%	17,6%	Не
`train-mixed-500`	500	Реальные + синтет.	68,0%	32,0%	Не

Синтетические датасеты (`train-synthetic-*`) сгенерированы по методологии Self-Instruct [18] и прошли процедуру гуманизации — автоматическое преобразование, приближающее стилистику и лексику запросов к реальным пользовательским формулировкам. Классовый баланс в синтетических датасетах поддерживается на уровне, близком к 50/50.

Датасет `train-real-250` содержит только реальные пользовательские запросы с естественным распределением классов (82,4% GEO / 17,6% TAG), что отражает доминирование географических запросов в предметной области.

Датасет `train-mixed-500` представляет собой объединение реальных и синтетических примеров, что позволяет исследовать эффект смешивания типов данных при обучении.

Выбор такого набора датасетов мотивирован необходимостью ответить на следующие исследовательские вопросы: (а) как объём обучающей выборки влияет на качество (серия `synthetic-50/250/500/1000`); (б) превосходят ли реальные данные синтетические при равном объёме (`real-250` vs `synthetic-250`); (в) даёт ли смешивание типов данных преимущество (`mixed-500` vs `synthetic-500`).

Помимо основных шести датасетов, были обучены дополнительные версии моделей `lite_latest` (v1, v2, v3) и `lite_rc` (v1, v2) на корпусе из приблизительно 1500 синтетических примеров. Эти модели представляют собой результаты итеративного улучшения конфигурации обучения.

5.4. Протокол дообучения

5.4.1. Метод дообучения

Дообучение выполняется методом LoRA (Low-Rank Adaptation) [11] через API Yandex Cloud [22]. Метод LoRA добавляет к замороженным весам предобученной модели пару низкоранговых матриц $A \in \mathbb{R}^{d \times r}$ и $B \in \mathbb{R}^{r \times d}$ (где $r \ll d$), что позволяет адаптировать модель при минимальном числе обучаемых параметров. Выходная матрица модифицированного слоя вычисляется как:

$$h = W_0 x + BAx,$$

где W_0 — замороженные веса исходной модели.

Yandex Cloud API предоставляет дообучение как управляемый сервис: пользователь загружает обучающий датасет в формате JSONL, указывает базовую модель и запускает задачу обучения. Внутренние гиперпараметры LoRA (ранг r, коэффициент масштабирования α, learning rate) управляются платформой и не доступны для прямого задания пользователем.

5.4.2. Базовая модель

В качестве базовой модели для дообучения используется YandexGPT Lite — компактная версия семейства YandexGPT, оптимизированная для задач с низкой латентностью. Для сравнения в роли baseline выступает более крупная модель YandexGPT 5.1 Pro (`pro_baseline`), используемая без дообучения.

5.4.3. Конфигурация инференса

При проведении инференса на бенчмарках для всех моделей применяются единые параметры генерации:

- **Temperature**: 0,1 — низкое значение для обеспечения детерминированности и воспроизводимости ответов;

- **Max tokens**: 4096 — верхняя граница длины генерируемого ответа;
- **Парсинг**: извлечение имени инструмента из XML-тегов `<tool_call>` в ответе модели.

5.5. Дизайн эксперимента

5.5.1. Перечень моделей

В эксперименте участвуют 12 моделей: 1 baseline-модель без дообучения и 11 дообученных вариантов YandexGPT Lite. Полный перечень приведён в Табл. 4.

Таблица 4: Перечень моделей, участвующих в эксперименте.

Модель	Базовая архитектура	Обучающий датасет	Объём
`pro_baseline`	YandexGPT 5.1 Pro	— (без дообучения)	—
`train_synth_50`	YandexGPT Lite	`train-synthetic-50`	50
`train_synth_250`	YandexGPT Lite	`train-synthetic-250`	250
`train_synth_500`	YandexGPT Lite	`train-synthetic-500`	500
`train_synth_1000`	YandexGPT Lite	`train-synthetic-1000`	908
`train_real_250`	YandexGPT Lite	`train-real-250`	250
`train_mixed_500`	YandexGPT Lite	`train-mixed-500`	500
`lite_latest_v1`	YandexGPT Lite	∼1500 синтет.	∼1500
`lite_latest_v2`	YandexGPT Lite	∼1500 синтет.	∼1500
`lite_latest_v3`	YandexGPT Lite	∼1500 синтет.	∼1500
`lite_rc_v1`	YandexGPT Lite	∼1500 синтет.	∼1500
`lite_rc_v2`	YandexGPT Lite	∼1500 синтет.	∼1500

5.5.2. Матрица экспериментов

Каждая из 12 моделей оценивается на каждом из 2 бенчмарков, что даёт $12 \times 2 = 24$ эксперимента. Для каждого эксперимента фиксируются бинарные исходы (верно/неверно) по каждому примеру бенчмарка,

что обеспечивает возможность применения попарных статистических тестов на связанных выборках.

5.5.3. Статистические тесты

Для анализа результатов применяется трёхуровневая схема статистического тестирования.

Глобальный тест. Критерий Cochran Q [3] используется для проверки нулевой гипотезы H_0: все k моделей имеют одинаковую вероятность правильной классификации. Критерий Cochran Q является обобщением критерия Макнемара на случай $k > 2$ связанных выборок и применим к бинарным исходам на одних и тех же объектах. Если H_0 отклоняется, переходим к попарным сравнениям.

Попарные сравнения. Для каждой пары моделей (i, j) применяется точный критерий Макнемара [12], основанный на таблице сопряжённости 2×2 дискордантных пар:

$$\chi^2 = \frac{(b-c)^2}{b+c},$$

где b — число примеров, классифицированных верно моделью i и неверно моделью j, а c — наоборот. При числе пар $C_k^2 = \binom{12}{2} = 66$ применяется поправка Бонферрони [2] для контроля семейной ошибки первого рода:

$$\alpha_{\text{adj}} = \frac{\alpha}{C_k^2} = \frac{0{,}05}{66} \approx 0{,}0008.$$

Доверительные интервалы. Для оценки accuracy каждой модели строятся 95%-доверительные интервалы методом бутстрапа [5] с параметрами $B = 1000$ (число бутстрап-выборок) и фиксированным зерном генератора случайных чисел seed $= 42$.

5.5.4. Контроль воспроизводимости

Для обеспечения воспроизводимости результатов приняты следующие меры:

- Фиксированное значение temperature = 0,1 при инференсе для минимизации стохастичности генерации.
- Фиксированное зерно seed = 42 для всех процедур бутстрапа.
- Сохранение уникальных идентификаторов чекпоинтов всех дообученных моделей в Yandex Cloud.
- Сохранение полных логов инференса (запрос, ответ модели, извлечённая метка) для каждого эксперимента.
- Использование единых бенчмарков без перемешивания или ресемплирования между запусками.

6. Результаты экспериментов и их анализ

В данном разделе приводятся результаты экспериментальной оценки 12 моделей на двух независимых бенчмарках: `benchmark-combined` (500 примеров: 326 GEO / 174 TAG) и `benchmark-real` (250 примеров: 203 GEO / 47 TAG). Методология подготовки данных и проведения экспериментов описана в разделе 5; определения метрик — в разделе 4.

6.1. Оценка базовой модели

В качестве базовой линии (baseline) используется модель YandexGPT 5.1 Pr без дообучения, обозначенная `pro_baseline`. Результаты представлены в таблице 5.

Таблица 5: Метрики базовой модели `pro_baseline` на двух бенчмарках

Бенчмарк	**Accuracy**	**Macro F1**	**Ошибки**	**No Pred**
combined (500)	0,818	0,526	57	34
real (250)	0,828	0,617	28	15

Низкое значение Macro $F_1 = 0{,}526$ при относительно высокой accuracy 0,818 указывает на выраженный дисбаланс качества между классами. Анализ матрицы ошибок выявляет систематический паттерн: все 57 ошибок на `benchmark-combined` представляют собой ошибки типа TAG→GEO (88 на уровне подробных категорий, включая `GetSearchQuerySummary`), то есть запросы класса TAG массово классифицируются как GEO. Ни одного обратного случая (GEO→TAG) не зафиксировано.

Данный bias объясняется тем, что базовая модель не дообучена для данного набора инструментов и по умолчанию предпочитает вызов `ResolveEntities` (GEO) как более общий. Помимо ошибок классификации, для 34 запросов (6,8%) модель не генерирует `tool_call` вовсе ($\rho_{\varnothing} = 6{,}8\%$), что является критическим сбоем в контексте AI-агента.

6.2. Основные результаты дообучения

Результаты всех 12 моделей на `benchmark-combined` приведены в таблице 6. Модели различаются по типу и объёму обучающих данных.

Таблица 6: Результаты на `benchmark-combined` (500 примеров: 326 GEO / 174 TAG)

Модель	**Обуч. данные**	**Acc**	**Macro F1**	**W-F1**	**Ош.**	**NP**
pro_baseline	— (без тюна)	0,818	0,526	0,829	57	34
lite_latest_v1	Синтет. ∼1500	0,928	0,925	0,930	34	2
lite_latest_v2	Синтет. ∼1500	0,984	0,985	0,986	6	2
lite_latest_v3	Синтет. ∼1500	0,966	0,966	0,968	15	2
lite_rc_v1	Синтет. ∼1500	0,980	0,981	0,982	8	2
lite_rc_v2	Синтет. ∼1500	0,974	0,974	0,976	11	2
train_synth_50	Синтет. 50	0,946	0,631	0,950	25	2
train_synth_250	Синтет. 250	0,946	0,945	0,948	25	2
train_synth_500	Синтет. 500	0,978	0,979	0,980	9	2
train_synth_1000	Синтет. 908	0,956	0,955	0,958	20	2
train_real_250	Реал. 250	0,952	0,949	0,953	22	2
train_mixed_500	Микс 500	0,994	0,996	0,996	1	2

Ключевые наблюдения:

1. **Скачок при минимальном объёме данных.** Даже 50 синтетических примеров повышают accuracy с 0,818 до 0,946 (+12,8 п.п.), подтверждая гипотезу 1: даже минимальный объём примеров радикально улучшает качество, что согласуется с гипотезой поверхностного выравнивания [8].

2. **Устранение отказов.** Все дообученные модели снижают $\rho_{\varnothing}$ с 6,8% (34 из 500) до 0,4% (2 из 500). Оставшиеся 2 отказа обусловлены safety-фильтром (см. 6.7).

3. **Абсолютный лидер.** Модель `train_mixed_500`, обученная на сбалансированной комбинации реальных и синтетических данных

(500 примеров), достигает accuracy 0,994 и Macro $F_1 = 0{,}996$ с единственной ошибкой классификации.

6.3. Валидация на реальных данных

Для оценки обобщающей способности моделей используется независимый бенчмарк `benchmark-real`, составленный из реальных пользовательских запросов. Результаты и разность с `benchmark-combined` приведены в таблице 7.

Таблица 7: Результаты на `benchmark-real` (250 примеров: 203 GEO / 47 TAG)

Модель	**Acc**	**Macro F1**	**Ош.**	**NP**	**ΔAcc**
pro_baseline	0,828	0,617	28	15	+0,010
lite_latest_v1	0,904	0,872	22	2	−0,024
lite_latest_v2	0,968	0,959	6	2	−0,016
lite_latest_v3	0,932	0,906	15	2	−0,034
lite_rc_v1	0,948	0,921	11	2	−0,032
lite_rc_v2	0,944	0,916	12	2	−0,030
train_synth_50	0,904	0,584	22	2	−0,042
train_synth_250	0,892	0,856	25	2	−0,054
train_synth_500	0,956	0,942	9	2	−0,022
train_synth_1000	0,912	0,882	20	2	−0,044
train_real_250	0,980	0,978	3	2	+0,028
train_mixed_500	0,988	0,991	1	2	−0,006

Столбец ΔAcc показывает разность accuracy на `benchmark-real` и `benchmark-combined`. Модели, обученные исключительно на синтетических данных, демонстрируют устойчивое снижение accuracy на реальных данных в диапазоне $-2{,}2 \ldots -5{,}4$ п.п. Это свидетельствует об ограниченной обобщающей способности синтетических данных: модели частично переобучаются на характерные паттерны генерации, отсутствующие в реальных запросах.

Модель `train_real_250`, обученная на 250 реальных примерах, напротив, демонстрирует прирост +2,8 п.п. при переходе на `benchmark-real`, что объяснимо совпадением распределений обучающей и тестовой выборок.

Модель `train_mixed_500` показывает минимальную деградацию: $\Delta\text{Acc} = -0{,}006$, что свидетельствует о высокой робастности, достигаемой за счёт сочетания разнородных обучающих данных.

6.4. Анализ кривой обучения

Для исследования зависимости качества от объёма обучающих данных (RQ2) обучена серия моделей на синтетических выборках объёмом 50, 250, 500 и 908 примеров. Результаты на `benchmark-combined` представлены в таблице 8.

Таблица 8: Кривая обучения: зависимость метрик от объёма синтетических данных

Объём	**Accuracy**	**Macro F1**
50	0,946	0,631
250	0,946	0,945
500	0,978	0,979
908	0,956	0,955

Кривая обучения обнаруживает нетривиальный немонотонный характер:

1. **Насыщение accuracy при малых объёмах.** При переходе от 50 к 250 примерам accuracy не изменяется (0,946), однако Macro F_1 скачкообразно возрастает с 0,631 до 0,945. Это означает, что 50 примеров достаточно для устранения основного bias базовой модели, но недостаточно для сбалансированного распознавания обоих классов.

2. **Пик на 500 примерах.** Максимальное качество среди синтетических моделей достигается при 500 примерах: accuracy = 0,978,

Macro $F_1 = 0{,}979$.

3. **Деградация при увеличении до 908 примеров.** Дальнейшее наращивание объёма приводит к снижению accuracy до 0,956 ($-2{,}2$ п.п.). Такой эффект объяснятся переобучением на артефакты синтетической генерации: с ростом объёма выборки доля повторяющихся шаблонов увеличивается, и модель начинает эксплуатировать статистические закономерности, специфичные для синтетического распределения.

6.5. Анализ типа обучающих данных

Для проверки гипотезы 2 сравним модели, обученные на данных различного типа при сопоставимых объёмах. Результаты сведены в таблице 9.

Таблица 9: Сравнение моделей по типу обучающих данных

Модель	**Тип данных**	**combined**		**real**	
		Acc	M-F1	Acc	M-F1
train_synth_500	Синтет. 500	0,978	0,979	0,956	0,942
train_real_250	Реал. 250	0,952	0,949	0,980	0,978
train_mixed_500	Микс 500	0,994	0,996	0,988	0,991

Наблюдается отчётливая специализация моделей по типу обучающих данных:

- `train_synth_500` превосходит `train_real_250` на `benchmark-combined` ($+2{,}6$ п.п. по accuracy), но уступает ей на `benchmark-real` ($-2{,}4$ п.п.).
- `train_real_250` показывает лучший результат на `benchmark-real` (0,980), но её accuracy на `benchmark-combined` ниже (0,952), поскольку синтетические примеры в тестовой выборке имеют иную структуру.

- `train_mixed_500` объединяет преимущества обоих типов данных и доминирует на обоих бенчмарках: 0,994 и 0,988 соответственно.

Данный результат подтверждает гипотезу 2: качество (разнородность и репрезентативность) обучающих данных оказывает большее влияние на итоговые метрики, чем их объём. Модель `train_mixed_500`, обученная на 500 примерах, превосходит все модели, обученные на ~ 1500 синтетических примерах.

6.6. Стабильность дообучения

Для проверки гипотезы 3 о нестабильности процесса дообучения через Yandex Cloud API проведены три независимых запуска обучения с идентичными данными (~ 1500 синтетических примеров) и гиперпараметрами. Результаты на `benchmark-combined` приведены в таблице 10.

Таблица 10: Стабильность дообучения: три независимых запуска на одних данных

Версия	**Accuracy**	**Macro F1**	**Ошибки**
lite_latest_v1	0,928	0,925	34
lite_latest_v2	0,984	0,985	6
lite_latest_v3	0,966	0,966	15

Разброс accuracy составляет 5,6 п.п. (от 0,928 до 0,984), а число ошибок варьируется от 6 до 34. Данный разброс является практически значимым: наихудший запуск (v1) допускает в 5,7 раза больше ошибок, чем наилучший (v2).

Подобная вариативность подтверждает гипотезу 3 и указывает на то, что процесс дообучения через Yandex Cloud API содержит стохастические компоненты (инициализация, порядок подачи батчей, стохастический градиентный спуск), не контролируемые пользователем. С практической точки зрения это означает необходимость проведения нескольких запусков обучения и выбора наилучшей модели по результатам валидации.

6.7. Анализ ошибок

6.7.1. Паттерны ошибок классификации

Для каждой модели определены направления ошибок: GEO→TAG (запрос GEO ошибочно классифицирован как TAG) и TAG→GEO (запрос TAG ошибочно классифицирован как GEO). Результаты представлены в таблице 11.

Таблица 11: Паттерны ошибок классификации на `benchmark-combined`

Модель	**GEO→TAG**	**TAG→GEO**	**Доминирующая ошибка**
pro_baseline	0	88	TAG массово как GEO
train_synth_*	8–24	1	GEO массово как TAG
train_real_250	1	21	TAG→GEO (дисбаланс)
train_mixed_500	0	1	Практически нет

Обнаруживается характерная инверсия bias: базовая модель систематически классифицирует TAG-запросы как GEO, тогда как дообученные на синтетике модели совершают преимущественно обратную ошибку (GEO→TAG). Это объясняется тем, что синтетические обучающие данные перепредставляют класс TAG, смещая решающую границу модели.

Модель `train_mixed_500` практически свободна от обоих типов bias (0 ошибок GEO→TAG, 1 ошибка TAG→GEO), что подтверждает эффективность сбалансированного микса данных для устранения систематических смещений.

6.7.2. Консервация ошибок

Анализ пересечения ошибок на `benchmark-combined` и `benchmark-real` выявляет важный феномен: многие ошибки воспроизводимы на обоих бенчмарках, что свидетельствует об их детерминированной природе (таблица 12).

Таблица 12: Консервация ошибок: совпадение ошибок на двух бенчмарках (combined / real)

Модель	**GEO→TAG**	**TAG→GEO**
lite_latest_v2	5 / 5	1 / 1
train_synth_500	8 / 8	1 / 1
train_mixed_500	0 / 0	1 / 1

Для модели `lite_latest_v2` все 5 ошибок типа GEO→TAG и 1 ошибка TAG→GEO на `benchmark-combined` совпадают с ошибками на `benchmark-re` Аналогичная картина наблюдается для `train_synth_500`. Это означает, что ошибки обусловлены не стохастикой генерации, а систематическими слабостями модели на определённых типах запросов («трудные» примеры).

6.7.3. Safety-фильтр

Два запроса (`q_00159` и `q_00253`) отклоняются всеми 12 моделями без исключения. Оба запроса содержат упоминание Крыма, что активирует safety-фильтр Yandex Cloud API. Эти 2 примера учтены в столбце «No Pred» (NP) всех дообученных моделей и не являются следствием ошибок классификации.

6.8. Статистическая значимость

6.8.1. Критерий Кохрена

Для проверки глобальной гипотезы о равенстве долей ошибок всех 12 моделей применён Q-критерий Кохрена (Cochran's Q test), являющийся обобщением критерия Макнемара на случай $k > 2$ связанных выборок (раздел 4):

- `benchmark-combined`: $Q = 348{,}0$, $p < 10^{-6}$;
- `benchmark-real`: $Q = 128{,}8$, $p < 10^{-6}$.

На обоих бенчмарках нулевая гипотеза $H_0\colon \pi_1 = \pi_2 = \ldots = \pi_{12}$ отвергается при любом разумном уровне значимости. Следовательно, модели статистически значимо различаются по доле ошибок.

6.8.2. Попарные сравнения (критерий Макнемара)

Для попарного сравнения моделей применён точный критерий Макнемара с поправкой Бонферрони на множественные сравнения. При 12 моделях число пар составляет $\binom{12}{2} = 66$, скорректированный уровень значимости $\alpha_{\text{adj}} = 0{,}05/66 \approx 0{,}0008$.

`benchmark-combined`.

- `train_mixed_500` статистически значимо превосходит большинство моделей ($p < \alpha_{\text{adj}}$ для всех попарных сравнений, кроме `lite_latest_v2`).
- Пара `lite_latest_v2` — `lite_rc_v1` неразличима: $p = 0{,}73 \gg \alpha_{\text{adj}}$.

`benchmark-real`.

- `train_mixed_500` и `train_real_250` статистически неразличимы: $p = 0{,}50$.
- Обе модели статистически значимо превосходят все модели, обученные исключительно на синтетических данных.

Данные результаты означают, что на реальных данных наличие реальных примеров в обучающей выборке является ключевым фактором: модель `train_real_250` (250 реальных примеров) статистически неотличима от `train_mixed_500` (500 смешанных), тогда как `train_synth_500` (500 синтетических) значимо уступает обеим.

6.8.3. Согласованность предсказаний

Анализ согласованности позволяет оценить долю примеров, на которых все 12 моделей дают одинаковый ответ:

- `benchmark-combined`: 368 из 500 примеров (73,6%) классифицированы правильно всеми 12 моделями; 0 примеров ошибочны у всех 12.

- `benchmark-real`: 175 из 250 примеров (70,0%) классифицированы правильно всеми 12 моделями.

Отсутствие примеров, ошибочных у всех 12 моделей, означает, что каждый запрос в бенчмарке в принципе решаем хотя бы одной из рассмотренных моделей. Оставшиеся $\sim$27–30% примеров составляют «зону разногласий», где модели расходятся в предсказаниях, и именно на этих примерах сосредоточены различия в качестве.

6.9. Ответы на исследовательские вопросы

RQ1: Каково влияние дообучения на метрики классификации? Дообучение методом LoRA приводит к статистически значимому улучшению всех метрик. Лучшая модель `train_mixed_500` повышает accuracy с 0,818 до 0,994 (+17,6 п.п.) и Macro F_1 с 0,526 до 0,996 (+0,470). Даже наименее успешная дообученная модель (`lite_latest_v1`, accuracy = 0,928) превосходит baseline на +11,0 п.п. Число отказов ($\rho_\varnothing$) снижается с 6,8% до 0,4% для всех дообученных моделей. Результат статистически значим по критерию Кохрена ($Q = 348{,}0$, $p < 10^{-6}$) и попарным критериям Макнемара.

RQ2: Как зависит качество от объёма обучающих данных? Зависимость немонотонна. При фиксированном типе данных (синтетика) accuracy растёт от 0,946 (50 примеров) до пика 0,978 (500 примеров), после чего снижается до 0,956 (908 примеров). Наиболее информативна динамика Macro F_1: скачок с 0,631 до 0,945 при переходе от 50 к 250 примерам указывает на то, что именно этот диапазон объёмов критичен для достижения сбалансированного распознавания обоих классов. Деградация при 908 примерах свидетельствует о переобучении на синтетическое распределение.

RQ3: Как тип обучающих данных влияет на робастность модели? Тип данных оказывает определяющее влияние. Модели на чистой синтетике демонстрируют деградацию $-2{,}2 \ldots -5{,}4$ п.п. при переходе на реальные данные. Модель `train_real_250` показывает обратный эффект: $+2{,}8$ п.п. на `benchmark-real` по сравнению с `benchmark-combined`. Оптимальная стратегия — смешанный датасет: `train_mixed_500` достигает accuracy $\geq 0{,}988$ на обоих бенчмарках с деградацией всего $-0{,}6$ п.п. Это подтверждает гипотезу 2.

RQ4: Статистически значимы ли различия между моделями? Да. Критерий Кохрена отвергает гипотезу о равенстве моделей на обоих бенчмарках ($p < 10^{-6}$). Попарные сравнения Макнемара с поправкой Бонферрони ($\alpha_{\text{adj}} \approx 0{,}0008$) показывают, что `train_mixed_500` значимо превосходит большинство моделей на `benchmark-combined`. На `benchmark-real` модели `train_mixed_500` и `train_real_250` статистически неразличимы ($p = 0{,}50$) и обе значимо лучше всех чисто синтетических моделей. Пара `lite_latest_v2` — `lite_rc_v1` также неразличима ($p = 0{,}73$), что подтверждает наблюдения о стохастичности процесса дообучения.

7. Программная реализация

Для проведения экспериментов по исследованию влияния дообучения большой языковой модели на качество классификации инструментов был разработан программный комплекс `ml_benchmark`. Система реализована на языке Python 3.11 и обеспечивает полный цикл экспериментальной работы: от генерации синтетических данных до статистического анализа результатов.

7.1. Архитектура системы

Программный комплекс построен по принципу модульной архитектуры с чётким разделением ответственности между слоями. Верхний уровень образует интерактивный CLI-интерфейс, реализованный с помощью библиотеки Rich и предоставляющий меню из 11 команд, каждая из которых соответствует отдельному этапу экспериментального процесса. Средний уровень составляет бизнес-логика, сосредоточенная в модуле `core/`: генерация данных, инференс моделей, вычисление метрик, статистический анализ, визуализация и формирование отчётов. Нижний уровень включает подсистемы хранения (`storage/`), конфигурации (`config/`) и LLM-клиентов (`clients/`).

Взаимодействие между слоями организовано через паттерн внедрения зависимостей (Dependency Injection). Главный цикл приложения, реализованный в классе `App` (модуль `cli.py`), при инициализации создаёт четыре ключевых объекта: `settings` (конфигурация из YAML-файлов и переменных окружения), `artifacts` (менеджер файловых артефактов), `registry` (реестр моделей) и `saves` (менеджер сохранений). Каждый обработчик команды получает эти объекты в качестве параметров, что обеспечивает слабую связанность компонентов и упрощает тестирование.

Для строгой типизации всех структур данных используются Pydantic-модели, определённые в пакете `models/`. Они описывают схемы датасетов, экспериментов, метрик, отчётов групповой оценки и итоговых отчётов. Валидация данных на уровне моделей позволяет обнаруживать

ошибки на ранних стадиях и гарантирует целостность данных при их передаче между компонентами.

Подсистема LLM-клиентов реализована по принципу plugin-архитектуры. Абстрактный базовый класс `BaseClient` определяет интерфейс взаимодействия с языковыми моделями, включая типы `Message` и `ChatResponse`. Конкретные реализации, такие как `YandexCloudClient`, наследуют этот интерфейс и инкапсулируют специфику работы с API конкретных провайдеров. Такой подход позволяет подключать новых провайдеров без модификации существующего кода.

7.2. Подсистема статистического анализа

Центральным компонентом подсистемы статистического анализа является класс `GroupEvaluator`, реализованный в модуле `core/group_evaluator` Данный класс обеспечивает статистически корректное сравнение нескольких моделей на одном и том же наборе тестовых примеров. Алгоритм его работы включает пять основных этапов.

На первом этапе выполняется построение бинарной матрицы результатов размерности $n_{\text{samples}} \times n_{\text{models}}$, где каждый элемент $b_{ij} \in \{0, 1\}$ отражает корректность предсказания j-й модели на i-м примере. Классификация предсказания как верного (TP, TN) или ошибочного (FP, FN) осуществляется модулем `Evaluator`, который выполняет парсинг вызовов инструментов из XML-ответов модели и сопоставляет их с эталонными значениями.

На втором этапе для каждой модели вычисляются доверительные интервалы метрик с помощью бутстрапа. Используется $B = 1000$ итераций ресемплирования с возвращением, что обеспечивает непараметрическую оценку распределения метрик без предположений о их аналитическом виде.

На третьем этапе выполняется проверка глобальной гипотезы H_0 о равенстве долей правильных ответов всех моделей с помощью критерия Кохрена Q (реализация из библиотеки `statsmodels`). Данный критерий является обобщением критерия Макнемара на случай $k > 2$ связанных

выборок и применим к бинарным данным при условии, что все модели оцениваются на одних и тех же примерах.

Четвёртый этап выполняется только при отклонении глобальной гипотезы и включает попарные сравнения моделей с помощью критерия Макнемара. Для контроля ошибки первого рода при множественных сравнениях применяется поправка Бонферрони, корректирующая уровень значимости пропорционально числу пар.

На пятом этапе проводится анализ ошибок: для каждой модели формируется перечень примеров с ошибочной классификацией, что позволяет выявить систематические паттерны ошибок и понять, на каких типах запросов модель испытывает затруднения.

7.3. Подсистема инференса

Подсистема инференса реализована в классе `InferenceRunner` (модуль `core/runner.py`) и обеспечивает запуск языковых моделей на наборе тестовых примеров с учётом ограничений внешних API.

Для обеспечения устойчивости к сбоям реализован механизм чекпоинтов: после обработки каждого примера промежуточные результаты сохраняются на диск. При повторном запуске система автоматически обнаруживает последний чекпоинт и продолжает выполнение с того примера, на котором произошло прерывание. Это критически важно при работе с большими датасетами, поскольку один полный прогон может занимать значительное время.

Для соблюдения ограничений частоты запросов (rate limits) внешних API между последовательными вызовами модели выдерживается пауза в 0.5 секунды. Кроме того, реализована стратегия повторных попыток (retry) при получении временных ошибок от API, что повышает надёжность сбора данных.

Парсинг ответов модели выполняется классом `ToolCallParser` (модуль `core/evaluator.py`), который извлекает вызовы инструментов из XML-разметки ответа. Извлечённые вызовы сопоставляются с эталонными, после чего класс `MetricsCollector` подсчитывает количество ис-

тинно положительных (TP), ложноположительных (FP), истинно отрицательных (TN) и ложноотрицательных (FN) результатов и на их основе вычисляет итоговые метрики классификации.

7.4. Подсистема генерации данных

Генерация синтетических тестовых данных выполняется двумя компонентами. Класс `Generator` (модуль `core/generator.py`) формирует синтетические пользовательские запросы с помощью вызовов LLM, обеспечивая контроль баланса классов в итоговом датасете. Класс `Humanizer` (модуль `core/humanizer.py`) выполняет «очеловечивание» сгенерированных запросов — повторно обрабатывает синтетические примеры через LLM, чтобы придать им стилистическое разнообразие, характерное для реальных пользовательских формулировок. Полученные данные конвертируются в формат JSONL классом `DatasetConverter` для последующего использования при дообучении моделей.

7.5. Обеспечение качества кода

Качество программного комплекса поддерживается комплексом автоматизированных инструментов. Юнит-тестирование осуществляется с помощью фреймворка `pytest`; покрытие кода тестами составляет порядка 85%. Статический анализ кода выполняется линтером `ruff`, а проверка типов — инструментом `mypy`. Совместное использование статической типизации `mypy` и валидации данных через Pydantic-модели во время исполнения обеспечивает двухуровневый контроль корректности: ошибки несовместимости типов обнаруживаются как на этапе разработки, так и при передаче данных между компонентами в процессе выполнения.

7.6. Технологический стек

Перечень основных библиотек и инструментов, используемых в проекте, приведён в таблице 13.

Таблица 13: Технологический стек проекта

Библиотека	**Назначение**
Python 3.11	Язык реализации
Pydantic	Описание и валидация схем данных
NumPy	Числовые операции, работа с матрицами результатов
statsmodels	Критерий Кохрена Q, критерий Макнемара
matplotlib	Построение графиков и визуализаций
Rich	Интерактивный CLI-интерфейс
PyYAML	Чтение конфигурационных файлов
pytest	Юнит-тестирование
ruff	Линтинг и форматирование кода
mypy	Статическая проверка типов

8. Заключение

В рамках настоящей работы проведено экспериментальное исследование влияния дообучения большой языковой модели YandexGPT 5.1 Pro методом LoRA [11] на качество классификации инструментов в AI-агенте, функционирующем в домене недвижимости. Исследование выполнено на двух независимых бенчмарках общим объёмом 750 примеров (500 смешанных и 250 реальных) и охватывает 12 моделей с вариацией типа и объёма обучающих данных.

Основные задачи, поставленные в начале исследования, были решены в полном объёме:

1. Формализована задача классификации инструментов как задача бинарной классификации с классами GEO (`ResolveEntities`) и TAG (`SearchTags`); сформированы два независимых бенчмарка для раздельной и совместной оценки моделей.

2. Разработан модульный программный пайплайн, обеспечивающий полный цикл — от подготовки обучающих данных и запуска дообучения через API YandexGPT до автоматизированного inference и вычисления метрик с визуализацией результатов.

3. Проведена серия из 12 экспериментов по дообучению с систематической вариацией источника данных (синтетические, реальные, смешанные), классового баланса и объёма выборки (от 50 до ~ 1500 примеров).

4. Выполнен строгий статистический анализ результатов с применением критерия Кохрена Q для проверки глобальной гетерогенности и попарного критерия Макнемара с поправкой Бонферрони для множественных сравнений.

Главные научные результаты работы заключаются в следующем:

- **Подтверждена гипотеза о стилистической природе выбора инструмента.** Дообучение методом LoRA повышает accuracy

с 81,8% (базовая модель) до 99,4% (лучшая дообученная модель `train_mixed_500`), а значение Macro F_1 — с 0,526 до 0,996. Прирост accuracy составляет +17,6 п. п., прирост Macro F_1 — +0,470. Различия статистически значимы: критерий Кохрена даёт $Q = 348{,}0$, $p < 10^{-6}$, а попарные сравнения по критерию Макнемара подтверждают значимость для всех пар «базовая — дообученная». Данный результат согласуется с гипотезой поверхностного выравнивания [8]: небольшой объём примеров корректного формата оказывается достаточным для радикального улучшения качества, поскольку модель уже обладает необходимыми знаниями о предметной области.

- **Установлена оптимальная стратегия формирования обучающей выборки.** Модель, дообученная на смешанном датасете из 250 реальных и 250 синтетических примеров (`train_mixed_500`), достигает accuracy 99,4%, превосходя модели, обученные на чисто синтетических данных: 97,8% для `synth_500` и 95,6% для `synth_1000`. Увеличение объёма синтетических данных сверх 500 примеров не только не улучшает, но ухудшает результат — наблюдается деградация до 95,6% при 908 синтетических примерах, что свидетельствует о переобучении на артефакты генерации. Наиболее робастной оказывается именно модель на смешанных данных: её отклонение на реальном бенчмарке составляет лишь −0,6 п. п., тогда как для моделей на чистой синтетике — от −2,2 до −5,4 п. п. Таким образом, качество обучающих данных оказывается важнее их объёма.

- **Обнаружена нестабильность процедуры дообучения.** Три версии модели (v1, v2, v3), дообученные на идентичных данных, демонстрируют разброс accuracy в 5,6 п. п. при неконтролируемой стохастичности, обусловленной отсутствием фиксации random seed на стороне API. Этот результат указывает на необходимость учёта случайной вариативности при оценке моделей и ставит под вопрос надёжность одиночных прогонов дообучения.

- **Выявлен эффект консервации ошибок и влияние safety-фильтра.** 73,6% примеров бенчмарка классифицируются одинаково всеми моделями, что указывает на устойчивое ядро «лёгких» и «трудных» запросов. Два запроса, содержащих упоминание Крыма, систематически отклоняются safety-фильтром YandexGPT вне зависимости от дообучения, что демонстрирует ограничения адаптации моделей с жёсткой модерацией контента.

Практическая значимость работы определяется следующим. Во-первых, разработанный программный пайплайн (описанный в разделе 7 и предшествующей работе [23]) может быть использован для дообучения и систематической оценки моделей YandexGPT в произвольных задачах классификации. Во-вторых, сформулированы конкретные рекомендации по формированию обучающих данных: использование смешанных выборок (реальные + синтетические данные) предпочтительнее наращивания объёма чисто синтетических примеров, что позволяет снизить затраты на ручную разметку без потери качества. В-третьих, показано, что даже для коммерческих моделей, доступных только через API, метод LoRA обеспечивает переход от неприемлемого уровня качества (81,8%) к практически безошибочной классификации (99,4%).

8.1. Направления дальнейших исследований

Проведённое исследование открывает ряд перспективных направлений для продолжения работы:

- **Переход к multi-label классификации.** В текущей постановке каждый запрос относится ровно к одному классу. Однако на практике встречаются запросы, содержащие одновременно географическую и атрибутивную составляющие (например, «трёхкомнатная квартира на Васильевском острове»). Расширение модели до multi-label классификации позволит обрабатывать такие случаи корректно.

- **Расширение бенчмарка и ревалидация разметки.** Текущие

бенчмарки общим объёмом 750 примеров (500 в смешанном и 250 в реальном), хотя и достаточны для статистического анализа, могут быть расширены для повышения мощности критериев и выявления более тонких различий между моделями. Кроме того, целесообразна независимая экспертная ревалидация разметки для устранения возможных ошибок в эталонных метках.

- **Стабилизация процедуры дообучения.** Обнаруженный разброс в 5,6 п. п. между версиями модели мотивирует исследование методов стабилизации: фиксацию random seed (при наличии такой возможности в API), ансамблирование нескольких прогонов дообучения, а также применение техник регуляризации.

- **Исследование альтернативных PEFT-методов.** В данной работе использован исключительно метод LoRA. Сравнение с другими подходами к параметрически эффективному дообучению — QLoRA, Prompt Tuning [10], Adapter Tuning — позволит определить, является ли LoRA оптимальным выбором для задачи tool selection.

- **Обобщение на другие LLM.** Все эксперименты проведены на модели YandexGPT 5.1 Pro. Проверка воспроизводимости результатов на моделях иных архитектур и провайдеров позволит установить, в какой мере полученные закономерности являются универсальными, а в какой — специфичными для конкретной модели.

Таким образом, настоящая работа вносит вклад в понимание механизмов адаптации больших языковых моделей к задаче выбора инструментов и демонстрирует, что дообучение методом LoRA на относительно небольшой, но качественно сформированной обучающей выборке является эффективной стратегией повышения качества классификации в AI-агентах.

Список литературы

[1] Attention Is All You Need / Ashish Vaswani, Noam Shazeer, Niki Parmar et al. // Advances in Neural Information Processing Systems. — Vol. 30. — 2017.

[2] Bonferroni Carlo. Teoria statistica delle classi e calcolo delle probabilità // Pubblicazioni del R Istituto Superiore di Scienze Economiche e Commericiali di Firenze. — 1936. — Vol. 8. — P. 3–62.

[3] Cochran William G. The Comparison of Percentages in Matched Samples // Biometrika. — 1950. — Vol. 37, no. 3/4. — P. 256–266.

[4] Dietterich Thomas G. Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms // Neural Computation. — 1998. — Vol. 10, no. 7. — P. 1895–1923.

[5] Efron Bradley. Bootstrap Methods: Another Look at the Jackknife // The Annals of Statistics. — 1979. — Vol. 7, no. 1. — P. 1–26.

[6] The False Promise of Imitating Proprietary LLMs / Arnav Gudibande, Eric Wallace, Charlie Snell et al. // arXiv preprint arXiv:2305.15717. — 2023.

[7] Gorilla: Large Language Model Connected with Massive APIs / Shishir G Patil, Tianjun Zhang, Xin Wang, Joseph E Gonzalez // arXiv preprint arXiv:2305.15334. — 2023.

[8] LIMA: Less Is More for Alignment / Chunting Zhou, Pengfei Liu, Puxin Xu et al. // Advances in Neural Information Processing Systems. — 2023. — Vol. 36.

[9] Language Models are Unsupervised Multitask Learners / Alec Radford, Jeffrey Wu, Rewon Child et al. // OpenAI blog. — 2019. — Vol. 1, no. 8. — P. 9.

[10] Lester Brian, Al-Rfou Rami, Constant Noah. The Power of Scale for Parameter-Efficient Prompt Tuning // arXiv preprint arXiv:2104.08691. — 2021.

[11] LoRA: Low-Rank Adaptation of Large Language Models / Edward J Hu, Yelong Shen, Phillip Wallis et al. // arXiv preprint arXiv:2106.09685. — 2022.

[12] McNemar Quinn. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages // Psychometrika. — 1947. — Vol. 12, no. 2. — P. 153–157.

[13] Overcoming Catastrophic Forgetting in Neural Networks / James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz et al. // Proceedings of the National Academy of Sciences. — 2017. — Vol. 114, no. 13. — P. 3521–3526.

[14] Parameter-Efficient Fine-Tuning of Large-Scale Pre-Trained Language Models / Ning Ding, Yujia Qin, Guang Yang et al. // Nature Machine Intelligence. — 2023. — Vol. 5. — P. 220–235.

[15] Parameter-Efficient Transfer Learning for NLP / Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski et al. // International Conference on Machine Learning / PMLR. — 2019. — P. 2790–2799.

[16] ReAct: Synergizing Reasoning and Acting in Language Models / Shunyu Yao, Jeffrey Zhao, Dian Yu et al. // arXiv preprint arXiv:2210.03629. — 2023.

[17] The Rise and Potential of Large Language Model Based Agents: A Survey / Zhiheng Xi, Wenxiang Chen, Xin Guo et al. // arXiv preprint arXiv:2309.07864. — 2023.

[18] Self-Instruct: Aligning Language Models with Self-Generated Instructions / Yizhong Wang, Yeganeh Kordi, Swaroop Mishra et al. // arXiv preprint arXiv:2212.10560. — 2023.

[19] Sokolova Marina, Lapalme Guy. A Systematic Analysis of Performance Measures for Classification Tasks // Information Processing & Management. — 2009. — Vol. 45, no. 4. — P. 427–437.

[20] Tool Learning with Foundation Models / Yujia Qin, Shengding Liang, Yining Ye et al. // arXiv preprint arXiv:2304.08354. — 2024.

[21] Toolformer: Language Models Can Teach Themselves to Use Tools / Timo Schick, Jane Dwivedi-Yu, Roberto Dessì et al. // arXiv preprint arXiv:2302.04761. — 2023.

[22] Yandex Cloud. YandexGPT API: Документация. — https://yandex.cloud/ru/docs/foundation-models/. — 2024. — Дата обращения: 2026-03-01.

[23] Горошко А. В. Исследование влияния дообучения большой языковой модели на качество классификации инструментов (Отчёт по учебной практике). — 2025. — СПбГУ, кафедра математики и компьютерных наук. Неопубликованная работа.